

Topic Matching in the Wild: Benchmark and Lessons from Real-World ASR Transcripts

Saman Rahbar, Xiliang Zhu, Irvin Cardoza, David Rossouw

Dialpad Inc.

{sam.rahbar, xzhu, irvin.cardoza, davidr}@dialpad.com

Abstract

In contact centers, real-time agent-assist tools determine, for each of many predefined topics, whether a live customer utterance is relevant and display a coaching card to the agent when it is. The input is noisy and challenging: ASR (Automatic Speech Recognition) transcripts of spontaneous phone conversations, which can be unclear, repetitive, and mostly lack punctuation. To systematically study this real-world task, we curate a human-annotated topic-utterance judgments dataset sourced from real call-center transcripts. We compare three types of matchers: a regex-based baseline, zero-shot sentence-embedding encoders, and Gemini-based LLM matchers. In addition, two types of topic representations are studied in our benchmark: keyphrases and natural language description. Our empirical experiments highlight the superior performance of lightweight LLM matchers over embedding and regex models when equipped with natural language descriptions.

1 Introduction

Contact centers can use real-time “agent-assist” software to support human agents during a live call. Streaming Automatic Speech Recognition (ASR) transcribes the customer, and based on a curated list of topics, the system decides whether any of them is present in the conversation. When a topic matches the current utterance, the system displays a coaching card for the agent (Fig. 1). This requires low-latency processing of hundreds of topics simultaneously (Rawat and Barres, 2022; Barrionuevo-Valenzuela et al., 2026). Additionally, the input text is a challenging ASR transcript of spontaneous speech containing disfluencies (e.g. “my, my, my, the, my vehicle”), false starts, and residual recognition errors. These affect downstream processing pipelines (Ritter et al., 2011), and the errors are non-negligible (Feng et al., 2021; Shapira et al., 2025). The standard approach is to clean the text,

for example, with lexical normalization (Han and Baldwin, 2011; van der Goot et al., 2021) or disfluency removal (Honnibal and Johnson, 2014; Zayats et al., 2016).

We consider the *binary topic-utterance matching* problem: given an utterance recognized with ASR, and a topic definition, decide whether the utterance matches the topic. To avoid additional latency, we process the raw ASR transcript. We compare two representations for the topic definition: the keyphrase lists that can be compiled into a regex matching program, and natural-language descriptions that can be processed by more advanced models. Both are evaluated on a human-annotated internal dataset. Various methods are benchmarked for matching utterances to topics: the regex baseline, zero-shot embedding similarity (Reimers and Gurevych, 2019), and Gemini LLM matchers (Gemini Team, 2025). We summarize our empirical contributions:

1. **C1: to our knowledge, the first study of topic matching on real, noisy production ASR, with the noise measured.** Previous resources are social-media text (Baldwin et al., 2015; Derczynski et al., 2017; van der Goot et al., 2021), read or synthetic spoken-language understanding (Bastianelli et al., 2020; Shon et al., 2022; Feng et al., 2021), or clean crowdsourced intent sets (Larson et al., 2019; Coucke et al., 2018; Casanueva et al., 2020). None target the noisy textual input setting. We measure the noise directly (utterance length, 11.4% disfluent repetitions, 24.2% filler markers). However, our transcripts are sensitive customer data, so we release the evaluation protocol and findings rather than the data.
2. **C2: an efficient Flash-tier LLM is enough.** The LLM matchers beat the regex baseline (the best by 12.6 F_1 points, $p < 10^{-14}$)

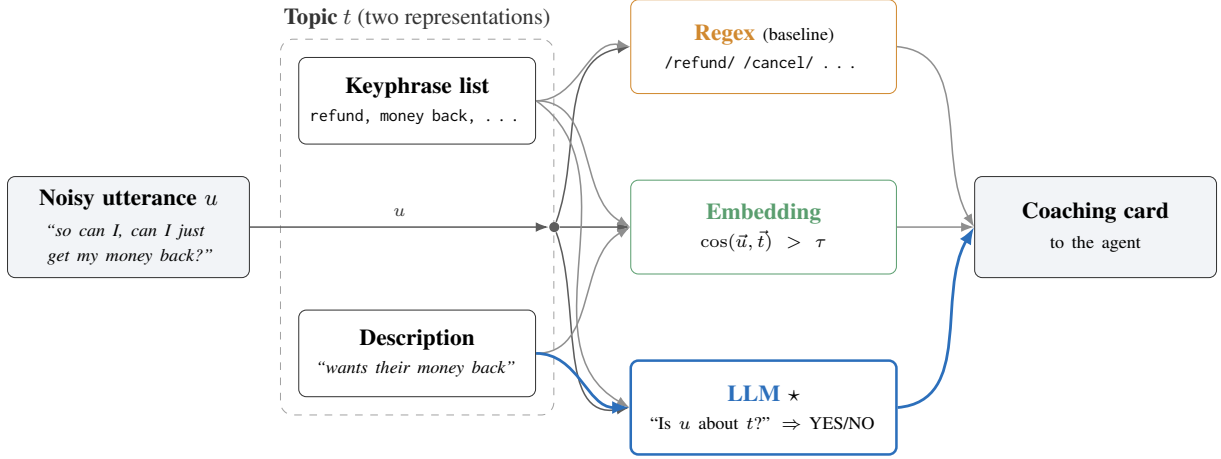


Figure 1: **Our evaluation space.** A live call yields a noisy utterance u ; a predefined topic t is expressed two ways: a **keyphrase list** or a **natural-language description**. Every matcher scores the utterance against a topic representation: a **regex** baseline (keyphrases only), a zero-shot **embedding** (cosine similarity), or an **LLM** prompted for a YES/NO decision. A positive match surfaces a coaching card to the agent. We evaluate every applicable (utterance, representation) pairing; the best ($\star$, blue) is an LLM reading a description.

and every zero-shot encoder, even though encoders are the standard tool for semantic matching. The best two matchers are Flash-tier (Gemini-3-Flash and Gemini-2.5-Flash-Lite), and the heavier (e.g. Gemini-2.5-Pro) does not beat them. So a small, fast model is the practical choice for a latency-bounded deployment.

3. **C3: the best way to represent and match a topic.** A topic can be written as a keyphrase list or as a natural-language description; we test every matcher against both and report the optimal combination. The strongest is an LLM reading a natural-language description ($F_1 = 0.847$), and the best representation differs by method—LLMs favor a description, embeddings a keyphrase list. How a topic is represented is thus as decisive as which matcher scores it.

2 Related Work

Noisy user-generated text and normalization. Applying standard NLP to non-canonical text is a long-standing problem. Non-standard vocabulary and syntax break tools trained on edited corpora (Eisenstein, 2013), and off-the-shelf pipelines degrade sharply on tweets (Ritter et al., 2011). The classic response is lexical normalization to a canonical vocabulary (Han and Baldwin, 2011), systems like MoNoise built on this idea (van der Goot and van Noord, 2017). Later work took normalization

multilingual (van der Goot et al., 2021). For spontaneous *speech*, the counterpart to orthographic normalization is disfluency detection and removal: finding repetitions, false starts, and self-corrections in transcribed speech (Honnibal and Johnson, 2014; Zayats et al., 2016). These are the phenomena our noise statistics quantify. Our setting is spontaneous customer speech through call-center ASR, not social-media orthography. Rather than normalizing first, we match topics directly to achieve optimal latency performance, and leave a normalize-then-match comparison to future work.

ASR-error robustness and SLU. A parallel line studies how transcription errors propagate downstream. Spoken language understanding (SLU) benchmarks push toward natural speech (Bastianelli et al., 2020; Shon et al., 2022). ASR-GLUE isolates natural language understanding (NLU) robustness under transcription noise (Feng et al., 2021). Models trained on clean text degrade on ASR output, and recovering that performance takes deliberate effort (Cui et al., 2021; Huang and Chen, 2020; Shapira et al., 2025). We differ in using real contact-center ASR, not read, with binary topic matching as the downstream task.

Zero-shot classification and semantic matching. Zero-shot prompting (Brown et al., 2020) with instruction tuning (Wei et al., 2022) underpins the LLM-based paradigm, in which strong models proxy human judgment (Zheng et al., 2023; Liu et al., 2023; Gilardi et al., 2023). As a non-

generative alternative we use zero-shot encoders (Reimers and Gurevych, 2019). Our set spans E5, BGE, MiniLM, and MPNet (Wang et al., 2022; Xiao et al., 2024; Wang et al., 2020; Song et al., 2020), chosen for their Massive Text Embedding Benchmark (MTEB) rankings (Muennighoff et al., 2023). To our best knowledge, no prior work systematically compares LLM matchers, zero-shot embeddings, and a regex baseline for semantic matching over noisy ASR.

Contact-center and intent-detection NLP. Our task is short-utterance intent detection. Canonical benchmarks define the family: CLINC150/OOS (Larson et al., 2019), Snips (Coucke et al., 2018), Banking77 (Casanueva et al., 2020), and HWU64 (Liu et al., 2019), alongside short-text classification (Li et al., 2020). These use clean crowdsourced text. Closer to deployment, prior work classifies intent from call-center transcripts (Zhong and Li, 2019) and triggers agent-assist over live support ASR (Rawat and Barres, 2022; Barrionuevo-Valenzuela et al., 2026). Related resources cover policy-driven dialogue (Chen et al., 2021) and multi-annotator customer service (Feigenblat et al., 2021). Our work combines these threads: an LLM matcher on noisy contact-center ASR for real-time agent assist, measured against a regex baseline and zero-shot embeddings.

3 Task and Data

3.1 Task formulation

We study *binary topic–utterance matching* for real-time contact center agent assist. During a live call, streaming ASR transcribes the customer, and for each predefined topic the system decides whether the current utterance¹ is *about* that topic. A positive decision surfaces a coaching card to the agent. This runs concurrently for many topics per call and is latency-sensitive (Rawat and Barres, 2022; Barrionuevo-Valenzuela et al., 2026). Formally, a matcher $f(u, d(t)) \rightarrow \{0, 1\}$ maps an utterance u and a topic definition $d(t)$ to fire or no-fire; soft-output models produce a score thresholded at τ . We score against human gold as binary classification, and because the classes are imbalanced (Section 3.2) we foreground precision, recall, and F_1 over accuracy. The task is a form of short-utterance

¹By *utterance* we mean a single finalized piece of ASR output—one stretch of the customer’s speech the recognizer has settled on, rather than a whole speaker turn or the entire call. We match each one on its own.

intent classification (Li et al., 2020; Larson et al., 2019; Casanueva et al., 2020; Coucke et al., 2018), but on unconstrained, noisy ASR.

Two topic representations. A topic can be represented two ways. The first is a **keyphrase list**: a curated set of trigger phrases based on filler-tolerant regexes. The second is a **natural-language description**: a short admin-authored sentence of when the topic applies, scored YES/NO by a model. This lets us ask which representation, paired with which matcher, works best, and whether free-text descriptions can replace hand-curated keyphrases.

Example. For a topic like *requesting a refund*, the keyphrase list holds triggers such as refund, money back, and reimburse, while the natural language description reads “The customer is asking to be refunded or to get their money back.” An utterance like “so can I, can I just get my money back for that?” should match; “my address is forty-two oak street” should not. (Examples are illustrative, not drawn from our real customer data.)

3.2 Data

Source. Our benchmark comes from real English customer-service calls at **11 companies** from various industries, covering **173 topics/coaching cards**. Across those cards the keyphrase lists hold **3,658 keyphrases** (2,447 distinct surface forms), and the keyphrase regex runs over the full set rather than a truncated sample.

Utterances are noisy ASR. Inputs are ASR transcripts of spontaneous customer speech. This is noisy user-generated text: disfluencies and repetitions, false starts, residual recognition errors, and no reliable sentence structure. These phenomena break off-the-shelf pipelines (Ritter et al., 2011; Han and Baldwin, 2011) and propagate into downstream understanding (Bastianelli et al., 2020; Feng et al., 2021; Shapira et al., 2025; Cui et al., 2021).

Noise characterization. Aggregate statistics over the utterances in the samples demonstrate the “noisy” nature of our data. The utterances are short and variable: mean 15.8 tokens, median 10, and 11.5% have ≤ 3 tokens. An immediate word repetition (a stuttered restart) appears in 11.4%, and 24.2% contain a filler or discourse marker (*uh, like, you know*; 2.5% of all tokens). These statistics quantify spontaneous-speech *disfluency*. True ASR recognition errors are also present, our ASR system

presents a word error rate (WER) of 14.26% for English.

3.3 Annotation and gold standard

Every (utterance, topic) pair is labeled independently by three internal annotators, who choose *match*, *no-match*, or *ambiguous* (guidelines in Appendix B). The third option is deliberate: a short, disfluent utterance is often genuinely underspecified, and forcing a binary choice would push that uncertainty into the gold as noise. The gold label is the majority vote, and a pair whose majority is *ambiguous*, or that has no majority, is set aside rather than guessed (73 pairs: 45 ambiguous, 28 without a majority). Of our annotated pairs, **2,655** carry a decisive positive or negative gold ($\sim 24\%$ positive) and form the evaluation set, which indicates the imbalance that motivates precision, recall, and F_1 over accuracy.

On the **2,655** fully voted pairs, inter-annotator agreement is Fleiss’ $\kappa = 0.660$ across the three categories, which translates to “substantial” on the Landis–Koch scale (Fleiss, 1971; Landis and Koch, 1977). It sits below perfect for a real reason: deciding whether a half-finished, disfluent utterance is *about* a topic is a judgment call that is hard for people, not only for models (Derczynski et al., 2017).

Together with authentic call center ASR, rather than the social media text (Baldwin et al., 2015; van der Goot et al., 2021) or the read and synthetic speech of SLU benchmarks (Bastianelli et al., 2020; Feng et al., 2021), our consensus-based, multi-annotator gold standard is what defines the new evaluation setting we contribute.

4 Methods

We evaluate three matcher families on one shared benchmark, the same 2,655-row human gold set from Section 3.2. The matcher families are (i) a regex baseline, (ii) zero-shot sentence-embedding encoders, and (iii) instruction-tuned large language model (LLM) matchers. Figure 1 summarizes each family, the topic representation it uses, and how it decides.

4.1 Regex baseline

Our baseline defines each topic by its keyphrase list and fires when an utterance matches any keyphrase under a word-boundary regex. The regex tolerates filler “slop” and normalizes hyphenation, punctuation, and numeric variants. We run it over the full

keyphrase set, which contains 3,658 keyphrases, 2,447 distinct, across 173 coaching cards, so the baseline reflects the keyphrase matcher as actually configured.

4.2 Zero-shot sentence-embedding encoders

We embed the utterance and the topic, then score their cosine similarity, following the Sentence-BERT encoder tradition (Reimers and Gurevych, 2019). We use six widely adopted encoders zero-shot, with no fine-tuning: e5-small/base-v2 (Wang et al., 2022), bge-small/base-en-v1.5 (Xiao et al., 2024), all-MiniLM-L6-v2 (Wang et al., 2020), and all-mpnet-base-v2 (Song et al., 2020). We selected them by their standing on the Massive Text Embedding Benchmark (MTEB) (Muennighoff et al., 2023). We run two modes. The *description* mode takes one cosine to the natural-language description. The *keyphrase max-similarity* mode takes the max cosine over the topic’s keyphrases. Each mode produces a continuous score that needs a threshold. Tuning that threshold on the test rows would bias the result, so we use 5-fold stratified cross-validation (CV) (Kohavi, 1995). We select the max- F_1 threshold on four folds, apply it to the held-out fold, and aggregate. The stratification preserves the $\sim 24\%$ positive rate in each fold.

4.3 LLM matchers

We recast matching as a prompted YES/NO decision for LLMs. Instruction tuning makes this viable zero-shot (Brown et al., 2020; Wei et al., 2022; Gilardi et al., 2023). We evaluate four Gemini matchers: Gemini-2.5-Flash-Lite, Gemini-2.5-Flash, Gemini-2.5-Pro, and Gemini-3-Flash (preview) (Gemini Team, 2023, 2024, 2025)—which together cover a range of speed and capability, the trade-off a real-time budget forces. All four use the same prompt (Appendix A) at temperature 0, and each emits a hard YES/NO that we use directly. A few responses come back unparseable (≤ 9 per Gemini-2.5 matcher, none for Gemini-3), and we treat these as no-match (fail-closed). We hold the prompt, decoding, and parsing fixed across all four isolates the model as the source of any performance gap.

4.4 Metrics

The classes are imbalanced. Following work on skewed evaluation (Saito and Rehmsmeier, 2015; Davis and Goadrich, 2006; Sokolova and Lapalme,

2009), we center precision, recall, and F_1 over accuracy. We score hard-output methods at their native YES/NO decision and embeddings at the CV threshold. We add several agreement and significance measures: 95% bootstrap confidence intervals (CIs) on F_1 ; McNemar tests for the headline contrasts; and F_1 on the fully adjudicated subset.

4.5 Cost and latency

For each matcher we measure per-decision latency and cost on 200 topic–utterance pairs sampled from the benchmark, discarding a short warm-up. Latency is the end-to-end time for one (utterance, topic) decision under identical conditions: for the LLM matchers, the round trip to the Google Gemini API (temperature 0, one YES/NO token); for embeddings, the utterance encode; for regex, the pattern match. We report it as a relative comparison, not a production service-level objective. We report the marginal API cost per 1,000 decisions for the LLM matchers, from the published per-token prices² applied to the measured input and output tokens (output includes billed thinking tokens). The underlying compute cost of serving any matcher (instance, CPU/GPU, memory) varies by deployment and is out of scope; we assume a comparable instance-serving cost across all approaches and report only the differential API cost, which the LLM matchers alone incur. The figures are therefore the additional per-decision API cost on top of a common serving baseline, not a total cost of ownership.

5 Results and Discussion

Figure 2 gives the headline: the best F_1 each matcher reaches. Tables 1 and 2 break the picture out by topic representation, so every matcher–representation pairing is visible. Because the classes are imbalanced ($\sim 24\%$ positive), we report precision, recall, and F_1 ; embeddings are scored at a cross-validated threshold and the other matchers at their native YES/NO decision, and unparseable LLM replies (at most nine per model) count as no-match.

5.1 An LLM on a description wins

The strongest matcher is an LLM reading a natural-language description. Gemini-3-Flash reaches $F_1 = 0.847$, ahead of the regex baseline (0.721) by 12.6 points and of the best embedding (0.708).

²Gemini API price list, accessed 2026-08-27.

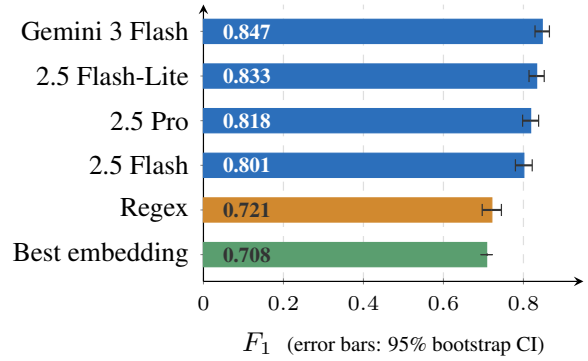


Figure 2: Main results: F_1 on the human-gold benchmark. Error bars are 95% bootstrap CIs for the native-decision methods; the **best embedding** bar is a 5-fold cross-validated point estimate (fold $\sigma = 0.027$; Table 1) and is shown without a bootstrap bar as it is not a native-decision method. **LLM matchers** (blue) lead the **regex** baseline (orange) and the best of six zero-shot embedding encoders (green); the two lightweight Flash-tier matchers top the board and the LLM–regex gap is significant ($p < 10^{-6}$ for all four matchers).

Keyphrases	P	R	F_1
Regex (baseline)	0.850	0.625	0.721
all-MiniLM-L6-v2	0.689	0.731	0.708
e5-base-v2	0.673	0.743	0.706
bge-base-en-v1.5	0.669	0.748	0.706
bge-small-en-v1.5	0.657	0.762	0.705
mpnet-base-v2	0.750	0.637	0.689
e5-small-v2	0.620	0.746	0.677
Gemini 3 Flash	0.839	0.752	0.793
2.5 Pro	0.845	0.725	0.780
2.5 Flash-Lite	0.882	0.638	0.740
2.5 Flash	0.898	0.603	0.721

Table 1: All matchers on the **keyphrase** representation. Embeddings are 5-fold cross-validated; regex and LLMs use their native decision. Best F_1 in bold.

The margin over regex is large and highly significant (exact McNemar $p < 10^{-6}$ for every LLM, and $p < 10^{-14}$ for the two best), and it is almost all recall: the LLM catches paraphrases and disfluent phrasings that a fixed keyphrase list misses. Surprisingly, embeddings come last though being the usual backbone of semantic matching in many other tasks.

Representation matters as much as the matcher.

Tables 1 and 2 show that the best representation is not the same for every method. LLMs are strongest on the description (Gemini-3-Flash 0.847 vs. 0.793 on keyphrases; Gemini-2.5-Flash-Lite 0.833 vs. 0.740); embeddings are strongest on keyphrases (0.708 vs. 0.660); regex applies only to keyphrases,

Natural-language description	P	R	F_1
e5-base-v2	0.687	0.635	0.660
all-MiniLM-L6-v2	0.660	0.640	0.649
bge-small-en-v1.5	0.646	0.640	0.641
bge-base-en-v1.5	0.615	0.657	0.635
e5-small-v2	0.646	0.616	0.627
mpnet-base-v2	0.604	0.626	0.613
Gemini 3 Flash	0.866	0.829	0.847
2.5 Flash-Lite	0.887	0.786	0.833
2.5 Pro	0.827	0.809	0.818
2.5 Flash	0.931	0.703	0.801

Table 2: All matchers on the **natural-language description** representation. Regex is omitted as it matches keyphrase lists only. Embeddings are 5-fold cross-validated. Best F_1 in bold.

since a description is not a list of patterns. The optimal pairing is therefore an LLM with a natural language description, and how a topic is represented is as consequential as which matcher scores it.

A lighter model is enough. The two best systems are the two smallest LLMs, Gemini-3-Flash and Gemini-2.5-Flash-Lite, and no larger model beats them. Gemini-2.5-Flash-Lite is even nominally ahead of the heavier Gemini-2.5-Pro and Gemini-2.5-Flash ($p = 0.014/0.019$). This interesting finding suggests that a small Flash-tier model is enough for this task while heavier, most costly LLMs offer no obvious benefits.

Cost and latency. Table 3 prices the “lighter is enough” finding. Gemini-2.5-Pro is Pareto-dominated: its mandatory thinking (~ 525 output tokens on a YES/NO question) makes it the slowest (p50 5.6 s) and by far the costliest (\$5.38 per 1,000 decisions), yet it does not lead on F_1 . Gemini-2.5-Flash-Lite sits at the opposite corner: p50 0.35 s and \$0.010 per 1,000, i.e. $\sim 16\times$ faster and $\sim 500\times$ cheaper than 2.5-Pro, at F_1 within 0.02 of the best matcher. Embeddings and regex are faster still and self-hosted, but trail on F_1 (Tables 1, 2). For a latency- and cost-bounded real-time deployment, a Flash-tier LLM on a description is the practical operating point.

5.2 Across three matchers

Our central finding is that LLM matchers score significantly higher on this noisy benchmark. What is clear mechanistically is that regex matches only literal keyphrases and cannot absorb disfluencies or recognition errors, and that cosine similarity rewards surface overlap rather than topic semantics

Matcher	Latency (ms)		Tokens		\$/1k
	p50	p95	in	out	
Gemini 3 Flash	1742	5018	100	165	0.55
2.5 Flash-Lite	346	528	100	1	0.010
2.5 Pro	5580	8365	100	525	5.38
2.5 Flash	461	771	100	1	0.033
e5-base-v2	29	32	—	—	—
all-MiniLM-L6-v2	6	7	—	—	—
Regex	0	0	—	—	—

Table 3: Per-decision latency and API cost on 200 sampled topic–utterance pairs. LLMs score the natural-language description; embedding latency is the per-utterance encode and regex the keyphrase match. Gemini \$/1k is the marginal API cost (prices accessed 2026-08-27; output includes billed thinking tokens). Embeddings and regex are self-hosted and incur no API cost. Latency is measured under identical conditions from one client to the Google Gemini API; embeddings and regex are timed locally on a single CPU core.

when processing noisy textual input as in our setting.

6 Conclusion

We studied topic–utterance matching on noisy call-center ASR transcripts, comparing a regex baseline, zero-shot embeddings, and Gemini LLM matchers across two ways of describing a topic. The clear winner is an LLM reading a natural-language description: Gemini-3-Flash reaches an F_1 of 0.847, well above the regex baseline (0.721) and the best embedding (≈ 0.71). Our comprehensive benchmark demonstrates that lightweight LLMs are enough for this task. The smallest Flash-tier matchers lead, while larger ones do not present superior performance. Additionally, we prove that the topic representation matters as much as the matcher: LLMs are strongest with a written description, while embeddings work better with a keyphrase list. Because the calls are sensitive customer data, we present this evaluation protocol and findings rather than the data itself. We hope this work provides insights to other practitioners working under a similar setting.

Limitations

Scope. All data are English and from a single domain, contact-center agent assist (11 companies, 173 coaching cards); we make no cross-lingual or cross-domain claims. This is an accuracy result: without a controlled clean-versus-noisy comparison, we do not isolate noise robustness from gen-

eral semantic capability.

Models. Our best matchers are proprietary Gemini API models (Gemini Team, 2025), and the very best (Gemini-3-Flash) is a preview model whose numbers are a snapshot. Our conclusions do not hinge on it: the generally available Gemini-2.5-Flash-Lite already beats every non-LLM baseline, and we anchor the comparison to a fully reproducible regex baseline and open-weight embeddings (Reimers and Gurevych, 2019). We do not test other LLM API services beyond Gemini due to our data safety policy.

Data availability. The underlying transcripts contain sensitive customer information and cannot be released. To support reproducibility within this constraint, we release the full evaluation protocol, prompts, matcher configurations, and aggregate statistics, and we report results only in aggregate; access to the data may be considered on a controlled basis subject to the service’s data-processing agreements.

Ethics and Broader Impact

The data is derived from customer–agent dialogues about sensitive topics and processed in accordance with the service’s data-processing and consent procedures; we use only the transcript text and do not seek to re-identify any customers or agents. We report only aggregated results, model performance scores, and evaluation methodology, not the data itself. Gold labels were obtained from internal specialized annotators who were compensated fairly.

References

- Timothy Baldwin, Marie Catherine de Marneffe, Bo Han, Young-Bum Kim, Alan Ritter, and Wei Xu. 2015. [Shared tasks of the 2015 workshop on noisy user-generated text: Twitter lexical normalization and named entity recognition](#). In *Proceedings of the Workshop on Noisy User-generated Text (WNUT 2015)*, ACL-IJCNLP.
- Juan Barrionuevo-Valenzuela, Daniel Calderón-González, Zoraida Callejas, and David Griol. 2026. [Supporting human operators during customer service interactions with agentic-rag](#). In *Proceedings of the 16th International Workshop on Spoken Dialogue System Technology (IWSDS 2026)*.
- Emanuele Bastianelli, Andrea Vanzo, Pawel Swietojanski, and Verena Rieser. 2020. [Slurp: A spoken language understanding resource package](#). *arXiv preprint arXiv:2011.13205*.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. [Language models are few-shot learners](#). *arXiv preprint arXiv:2005.14165*.
- Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. 2020. [Efficient intent detection with dual sentence encoders](#). *arXiv preprint arXiv:2003.04807*.
- Derek Chen, Howard Chen, Yi Yang, Alexander Lin, and Zhou Yu. 2021. [Action-based conversations dataset: A corpus for building more in-depth task-oriented dialogue systems](#). *arXiv preprint arXiv:2104.00783*.
- Alice Coucke, Alaa Saade, Adrien Ball, Théodore Bluche, Alexandre Caulier, David Leroy, Clément Doumouro, Thibault Gisselbrecht, Francesco Caltagirone, Thibaut Lavril, Maël Primet, and Joseph Dureau. 2018. [Snips voice platform: an embedded spoken language understanding system for private-by-design voice interfaces](#). *arXiv preprint arXiv:1805.10190*.
- Tong Cui, Jinghui Xiao, Liangyou Li, Xin Jiang, and Qun Liu. 2021. [An approach to improve robustness of nlp systems against asr errors](#). *arXiv preprint arXiv:2103.13610*.
- Jesse Davis and Mark Goadrich. 2006. [The relationship between precision-recall and roc curves](#). In *Proceedings of the 23rd International Conference on Machine Learning (ICML 2006)*, pp. 233-240.
- Leon Derczynski, Eric Nichols, Marieke van Erp, and Nut Limsopatham. 2017. [Results of the wnut2017 shared task on novel and emerging entity recognition](#). In *Proceedings of the 3rd Workshop on Noisy User-generated Text (W-NUT 2017)*.
- Jacob Eisenstein. 2013. [What to do about bad language on the internet](#). In *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2013)*.
- Guy Feigenblat, Chulaka Gunasekara, Benjamin Szneider, Sachindra Joshi, David Konopnicki, and Ranit Aharonov. 2021. [Tweetsumm – a dialog summarization dataset for customer service](#). *arXiv preprint arXiv:2111.11894*.
- Lingyun Feng, Jianwei Yu, Deng Cai, Songxiang Liu, Haitao Zheng, and Yan Wang. 2021. [Asr-glue: A new multi-task benchmark for asr-robust natural language understanding](#). *arXiv preprint arXiv:2108.13048*.
- Joseph L. Fleiss. 1971. [Measuring nominal scale agreement among many raters](#). *Psychological Bulletin* 76(5):378-382.

- Gemini Team. 2023. [Gemini: A family of highly capable multimodal models](#). *arXiv preprint arXiv:2312.11805*.
- Gemini Team. 2024. [Gemini 1.5: Unlocking multi-modal understanding across millions of tokens of context](#). *arXiv preprint arXiv:2403.05530*.
- Gemini Team. 2025. [Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities](#). *arXiv preprint arXiv:2507.06261*.
- Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. [Chatgpt outperforms crowd workers for text-annotation tasks](#). *arXiv preprint arXiv:2303.15056*.
- Bo Han and Timothy Baldwin. 2011. [Lexical normalisation of short text messages: Makn sens a #twitter](#). In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies (ACL-HLT 2011)*.
- Matthew Honnibal and Mark Johnson. 2014. Joint incremental disfluency detection and dependency parsing. *Transactions of the Association for Computational Linguistics*, 2:131–142.
- Chao-Wei Huang and Yun-Nung Chen. 2020. [Learning asr-robust contextualized embeddings for spoken language understanding](#). *arXiv preprint arXiv:1909.10861*.
- Ron Kohavi. 1995. [A study of cross-validation and bootstrap for accuracy estimation and model selection](#). In *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI 1995), Volume 2*, pp. 1137–1143.
- J. Richard Landis and Gary G. Koch. 1977. [The measurement of observer agreement for categorical data](#). *Biometrics* 33(1):159–174.
- Stefan Larson, Anish Mahendran, Joseph J. Peper, Christopher Clarke, Andrew Lee, Parker Hill, Jonathan K. Kummerfeld, Kevin Leach, Michael A. Laurenzano, Lingjia Tang, and Jason Mars. 2019. [An evaluation dataset for intent classification and out-of-scope prediction](#). *arXiv preprint arXiv:1909.02027*.
- Qian Li, Hao Peng, Jianxin Li, Congying Xia, Renyu Yang, Lichao Sun, Philip S. Yu, and Lifang He. 2020. [A survey on text classification: From shallow to deep learning](#). *arXiv preprint arXiv:2008.00364*.
- Xingkun Liu, Arash Eshghi, Pawel Swietojanski, and Verena Rieser. 2019. [Benchmarking natural language understanding services for building conversational agents](#). *arXiv preprint arXiv:1903.05566*.
- Yang Liu, Dan Iter, Yichong Xu, Shuhang Wang, Ruochen Xu, and Chenguang Zhu. 2023. [G-eval: Nlg evaluation using gpt-4 with better human alignment](#). *arXiv preprint arXiv:2303.16634*.
- Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. 2023. [Mteb: Massive text embedding benchmark](#). *arXiv preprint arXiv:2210.07316*.
- Mrinal Rawat and Victor Barres. 2022. [Real-time caller intent detection in human-human customer support spoken conversations](#). *arXiv preprint arXiv:2208.06802*.
- Nils Reimers and Iryna Gurevych. 2019. [Sentence-bert: Sentence embeddings using siamese bert-networks](#). *arXiv preprint arXiv:1908.10084*.
- Alan Ritter, Sam Clark, Mausam, and Oren Etzioni. 2011. [Named entity recognition in tweets: An experimental study](#). In *Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Takaya Saito and Marc Rehmsmeier. 2015. [The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets](#). *PLOS ONE* 10(3):e0118432.
- Ori Shapira, Shlomo E. Chazan, and Amir DN Cohen. 2025. [Measuring the effect of transcription noise on downstream language understanding tasks](#). *arXiv preprint arXiv:2502.13645*.
- Suwon Shon, Ankita Pasad, Felix Wu, Pablo Brusco, Yoav Artzi, Karen Livescu, and Kyu J. Han. 2022. [Slue: New benchmark tasks for spoken language understanding evaluation on natural speech](#). *arXiv preprint arXiv:2111.10367*.
- Marina Sokolova and Guy Lapalme. 2009. [A systematic analysis of performance measures for classification tasks](#). *Information Processing & Management* 45(4):427–437.
- Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. 2020. [Mpnet: Masked and permuted pre-training for language understanding](#). *arXiv preprint arXiv:2004.09297*.
- Rob van der Goot, Alan Ramponi, Arkaitz Zubiaga, Barbara Plank, Benjamin Muller, Iñaki San Vicente Roncal, Nikola Ljubešić, Özlem Çetinoğlu, Rahmad Mahendra, Talha Çolakoğlu, Timothy Baldwin, Tommaso Caselli, and Wladimir Sidorenko. 2021. [Multilexnorm: A shared task on multilingual lexical normalization](#). In *Proceedings of the Seventh Workshop on Noisy User-generated Text (W-NUT 2021)*.
- Rob van der Goot and Gertjan van Noord. 2017. [Monoise: Modeling noise using a modular normalization system](#). *arXiv preprint arXiv:1710.03476*.
- Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. [Text embeddings by weakly-supervised contrastive pre-training](#). *arXiv preprint arXiv:2212.03533*.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. [Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers](#). *arXiv preprint arXiv:2002.10957*.

Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. 2022. [Finetuned language models are zero-shot learners](#). *arXiv preprint arXiv:2109.01652*.

Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muenighoff, Defu Lian, and Jian-Yun Nie. 2024. [C-pack: Packed resources for general chinese embeddings](#). *arXiv preprint arXiv:2309.07597*.

Vicky Zayats, Mari Ostendorf, and Hannaneh Hajishirzi. 2016. Disfluency detection using a bidirectional LSTM. In *Interspeech*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhaghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. [Judging llm-as-a-judge with mt-bench and chatbot arena](#). *arXiv preprint arXiv:2306.05685*.

Junmei Zhong and William Li. 2019. [Predicting customer call intent by analyzing phone call transcripts based on cnn for multi-class classification](#). *arXiv preprint arXiv:1907.03715*.

A Evaluation Prompt

All LLM matchers receive the identical prompt below (temperature 0). The system instruction fixes the task and the strict YES/NO output format; the user template is filled per example with the topic description and the customer utterance.

System instruction.

You are a precise judge evaluating whether a customer utterance matches a topic description. The topic description defines what a customer might say when they are discussing that topic. You must reply with exactly one word: YES or NO. Do not add any explanation.

User template.

Topic description:
{description}
Customer utterance:
“{utterance}”
Does the utterance match the topic?
Reply YES or NO.

B Annotation Guidelines

Each item shows one customer utterance and one topic (its natural-language description). Up to three annotators independently answer a single question, presented verbatim as:

Does the utterance fit the description? Would you say the utterance fits under the topic described?

with response options **yes** (positive / match), **no** (negative / no-match), or **ambiguous** when the utterance is too underspecified to decide, plus an optional free-text notes field for rationale. Annotators judge *semantic* fit—whether the customer is talking about the topic—rather than surface keyword overlap, and are instructed to rely only on the utterance itself (no surrounding call context). The per-item gold label is the majority vote over the available annotators. Rows whose majority label is **ambiguous**, or with no majority (**disputed**), are excluded from the usable evaluation set; rows with only a single annotator so far are used as per-row gold but flagged and excluded from all inter-annotator agreement statistics (Section 3.2).